



Bikkad IT Institute

Session 7: Class & Object (OOP-1)

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Python Programming - Beginner Level

Notes

Prepared for: Bikkad IT Institute Students

Topic: OOP Part 1 - Fundamentals

Date: February 2026

1. What is OOP?

Object-Oriented Programming

OOP is a programming paradigm based on the concept of **objects** that contain data and code.

Key Benefits:

- Code reusability
- Organized and modular code
- Easy to maintain and modify
- Real-world modeling

Four Pillars of OOP:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

2. What is Object?

Definition

An object is an **instance of a class** containing data (attributes) and methods (functions).

Real-World Examples:

Object	Attributes	Methods
Car	color, model, price	start(), stop(), drive()
Student	name, age, marks	study(), attend()
Phone	brand, model, price	call(), message()

Table 1: Real-World Object Examples

Code Example:

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Object has attributes (data)

```
student1_name = "Rahul"  
student1_age = 25  
student1_marks = 85
```

Object has methods (behavior)

```
def student1_study():  
    print("Rahul is studying")
```

3. Class & Object

What is a Class?

A class is a **blueprint or template** for creating objects.

Syntax:

```
class ClassName:  
    # attributes  
    # methods
```

Code Examples

Example 1: Basic Class and Object

Define class

```
class Student:  
    pass
```

Create object (instance)

```
student1 = Student()  
student2 = Student()  
  
print(type(student1)) # <class 'main.Student'>
```

Example 2: Class with Attributes

```
class Student:  
    # Class attribute (shared by all objects)  
    school = "Bikkad IT Institute"
```

```
# Instance attributes (unique to each object)  
def __init__(self, name, age):
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
self.name = name  
self.age = age
```

Create objects

```
student1 = Student("Rahul", 25)  
student2 = Student("Priya", 22)  
  
print(student1.name) # Rahul  
print(student2.name) # Priya  
print(student1.school) # Bikkad IT Institute
```

Example 3: Class with Methods

```
class Student:  
    def init(self, name, marks):  
        self.name = name  
        self.marks = marks
```

```
# Instance method  
def display_info(self):  
    print(f"Name: {self.name}")  
    print(f"Marks: {self.marks}")
```

Create and use object

```
student1 = Student("Rahul", 85)  
student1.display_info()
```

Example 4: Multiple Objects

```
class Car:  
    def init(self, brand, model, price):  
        self.brand = brand  
        self.model = model  
        self.price = price
```

```
def display(self):  
    print(f"{self.brand} {self.model} - Rs.{self.price}")
```

OUR LOCATIONS

Create multiple objects

```
car1 = Car("Maruti", "Swift", 700000)
car2 = Car("Hyundai", "Creta", 1200000)
car3 = Car("Tata", "Nexon", 900000)

car1.display()
car2.display()
car3.display()
```

4. Constructor (Special Function)

What is Constructor?

A constructor is a **special method** that automatically executes when an object is created.

In Python: Constructor is defined using `__init__()` method.

Purpose:

- Initialize object attributes
- Allocate memory
- Set default values
- Execute setup code

Code Examples

Example 1: Constructor Basics

```
class Student:
# Constructor
def init(self):
print("Constructor called!")
print("Object created")
```

Constructor executes automatically

```
student1 = Student()
```

Output: Constructor called!

Object created

Example 2: Constructor with Parameters

```
class Student:
def init(self, name, age, marks):
self.name = name
self.age = age
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
self.marks = marks
print(f"Student {name} created")

student1 = Student("Rahul", 25, 85)
student2 = Student("Priya", 22, 90)
```

Example 3: Constructor with Default Values

```
class Student:
    def __init__(self, name, age=18, city="Nashik"):
        self.name = name
        self.age = age
        self.city = city

student1 = Student("Rahul")
student2 = Student("Priya", 22, "Mumbai")

print(student1.age) # 18 (default)
print(student2.age) # 22
```

Example 4: Constructor for Calculations

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
        # Calculate area in constructor
        self.area = length * width

rect1 = Rectangle(10, 5)
print(f"Area: {rect1.area}") # Area: 50
```

5. Method vs Functions

Feature	Function	Method
Definition	Independent block	Inside class
First Parameter	Any or none	self (for instance methods)
Access	Direct call	Object.method()
Association	Not tied to class	Tied to class/object

Table 2: Function vs Method

Code Examples

Example 1: Function (Outside Class)**OUR LOCATIONS**

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Regular function

```
def calculate_area(length, width):  
    return length * width
```

Call function directly

```
result = calculate_area(10, 5)  
print(result) # 50
```

Example 2: Method (Inside Class)

```
class Rectangle:  
    def init(self, length, width):  
        self.length = length  
        self.width = width
```

```
# Method  
def calculate_area(self):  
    return self.length * self.width
```

Need object to call method

```
rect = Rectangle(10, 5)  
result = rect.calculate_area()  
print(result) # 50
```

Example 3: Types of Methods

```
class Student:  
    school = "Bikkad IT" # Class attribute
```

```
# Instance method (uses self)  
def instance_method(self):  
    print(f"Instance: {self.name}")  
  
# Class method (uses cls)  
@classmethod  
def class_method(cls):  
    print(f"School: {cls.school}")  
  
# Static method (no self or cls)  
@staticmethod
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
def static_method():
    print("This is a static method")
```

6. Magic Method / Dunder Method

What are Magic Methods?

Special methods with **double underscores** before and after name. Python calls them automatically.

Common Magic Methods:

Method	Purpose
<code>__init__()</code>	Constructor
<code>__str__()</code>	String representation
<code>__len__()</code>	Length
<code>__add__()</code>	Addition (+)
<code>__eq__()</code>	Equality (==)
<code>__lt__()</code>	Less than (<)
<code>__del__()</code>	Destructor

Table 3: Common Magic Methods

Code Examples

Example 1: `__str__()` Method

```
class Student:
    def init(self, name, marks):
        self.name = name
        self.marks = marks
```

```
# String representation
def __str__(self):
    return f"Student: {self.name}, Marks: {self.marks}"
```

```
student1 = Student("Rahul", 85)
print(student1) # Student: Rahul, Marks: 85
```

Example 2: `__len__()` Method

```
class Classroom:
    def init(self, students):
        self.students = students
```

OUR LOCATIONS

```
def __len__(self):  
    return len(self.students)
```

```
class1 = Classroom(["Rahul", "Priya", "Amit"])  
print(len(class1)) # 3
```

Example 3: __add__ Method

```
class Number:  
def init(self, value):  
    self.value = value
```

```
def __add__(self, other):  
    return Number(self.value + other.value)  
  
def __str__(self):  
    return str(self.value)
```

```
num1 = Number(10)  
num2 = Number(20)  
result = num1 + num2 # Calls add()  
print(result) # 30
```

Example 4: __eq__ Method

```
class Student:  
def init(self, name, marks):  
    self.name = name  
    self.marks = marks
```

```
def __eq__(self, other):  
    return self.marks == other.marks
```

```
student1 = Student("Rahul", 85)  
student2 = Student("Priya", 85)  
student3 = Student("Amit", 90)  
  
print(student1 == student2) # True  
print(student1 == student3) # False
```

Example 5: __del__ Method

```
class Student:  
def init(self, name):  
    self.name = name  
    print(f"{name} created")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
def __del__(self):  
    print(f"{self.name} deleted")
```

```
student1 = Student("Rahul")  
del student1 # Calls del()
```

7. When to Use Constructor

Use Constructor When:

- Initializing attributes with values
- Setting default values
- Performing initial calculations
- Validating input data
- Connecting to resources (files, databases)
- Counting object instances

Code Examples

Example 1: Initialize Attributes

```
class Student:  
    def init(self, name, age, marks):  
        self.name = name  
        self.age = age  
        self.marks = marks
```

```
student1 = Student("Rahul", 25, 85)
```

Example 2: Set Default Values

```
class BankAccount:  
    def init(self, name, initial_balance=0):  
        self.name = name  
        self.balance = initial_balance  
        self.account_number = self.generate_account_number()
```

```
def generate_account_number(self):  
    import random  
    return random.randint(100000, 999999)
```

```
account1 = BankAccount("Rahul")  
account2 = BankAccount("Priya", 5000)
```

Example 3: Validation in Constructor

```
class Student:  
    def init(self, name, age, marks):  
        # Validate data  
        if age < 0:
```

OUR LOCATIONS

```
raise ValueError("Age cannot be negative")
if marks < 0 or marks > 100:
raise ValueError("Marks must be 0-100")
```

```
self.name = name
self.age = age
self.marks = marks
```

Valid

```
student1 = Student("Rahul", 25, 85)
```

Invalid - raises error

```
student2 = Student("Priya", -5, 90)
```

Example 4: Count Object Instances

```
class Student:
count = 0 # Class attribute
```

```
def __init__(self, name):
    self.name = name
    Student.count += 1 # Increment count

@classmethod
def get_count(cls):
    return cls.count
```

```
student1 = Student("Rahul")
student2 = Student("Priya")
student3 = Student("Amit")

print(Student.get_count()) # 3
```

8. Concept of Self

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

What is self?

`self` represents the **current instance** of the class. It's used to access attributes and methods.

Key Points:

- First parameter of instance methods
- Refers to the object calling the method
- Name can be anything, but `self` is convention
- Python passes it automatically

Code Examples

Example 1: Basic self Usage

```
class Student:
def init(self, name, age):
self.name = name # self refers to current object
self.age = age
```

```
def display(self):
    print(f"Name: {self.name}") # Access via self
    print(f"Age: {self.age}")
```

```
student1 = Student("Rahul", 25)
student1.display() # self = student1
```

Example 2: self in Multiple Methods

```
class Calculator:
def init(self, num1, num2):
self.num1 = num1
self.num2 = num2
```

```
def add(self):
    return self.num1 + self.num2

def multiply(self):
    return self.num1 * self.num2

def display_result(self):
    # Calling other methods using self
    print(f"Sum: {self.add()}")
    print(f"Product: {self.multiply()}")
```

```
calc = Calculator(10, 5)
calc.display_result()
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: self vs Other Parameter Names

class Student:

You can use any name, but self is convention

def **init**(myself, name, age):

myself.name = name

myself.age = age

```
def display(this):  
    print(f"{this.name} - {this.age}")
```

student1 = Student("Rahul", 25)

student1.display()

Example 4: Why self is Needed

class Student:

def **init**(self, name, marks):

Without self - creates local variables

name = name # Wrong!

```
# With self - creates object attributes  
self.name = name  
self.marks = marks
```

student1 = Student("Rahul", 85)

student2 = Student("Priya", 90)

Each object has its own attributes

print(student1.name) # Rahul

print(student2.name) # Priya

Example 5: self Distinguishes Objects

class Counter:

def **init**(self):

self.count = 0

```
def increment(self):  
    self.count += 1
```

```
def get_count(self):  
    return self.count
```

counter1 = Counter()

counter2 = Counter()

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajnagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
counter1.increment()  
counter1.increment()  
counter2.increment()  
  
print(counter1.get_count()) # 2  
print(counter2.get_count()) # 1
```

Practice Exercises

Exercise 1: Create Class

Create a Book class with title, author, and price. Add method to display book details.

Exercise 2: Constructor

Create a Circle class with radius. Calculate area and circumference in constructor.

Exercise 3: Multiple Objects

Create 3 Student objects with different data and display all their information.

Exercise 4: Magic Methods

Implement `__str__()` and `__eq__()` methods in a Product class.

Exercise 5: Validation

Create a BankAccount class with constructor that validates initial balance.

Exercise 6: Instance Counter

Create a class that counts how many objects have been created.

Exercise 7: Methods

Create a Rectangle class with methods to calculate area and perimeter.

Exercise 8: Self Practice

Create a class with multiple methods that call each other using self.

Quick Reference

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Concept	Syntax
Class	<code>class ClassName:</code>
Object	<code>obj = ClassName()</code>
Constructor	<code>def __init__(self):</code>
Method	<code>def method_name(self):</code>
Attribute	<code>self.attribute = value</code>
Call Method	<code>obj.method_name()</code>
Magic Method	<code>def __str__(self):</code>

Table 4: OOP Quick Reference

Key Concepts Summary

Class vs Object:

- Class is blueprint, object is instance
- One class, many objects
- Class defines structure, objects hold data

Constructor Rules:

- Always named `__init__()`
- First parameter is `self`
- Called automatically on object creation
- Can have parameters and default values

self Importance:

- Refers to current object instance
- Required for accessing attributes and methods
- Python convention (can use other names)
- Passed automatically by Python

End of Notes

Prepared by: Bikkad IT Institute

For: Python Beginner Level Students

Date: February 2026

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune